IR Agent: An AI Agent Extended with Information Retrieval Methods

Belén Remedi, May 2026

Abstract

This paper describes IR Agent, an AI agent extended with Information Retrieval (IR) methods to improve its ability to find relevant context when answering questions and performing tasks. The system implements BM25 document retrieval with chunking, LLM-based query expansion, persistent working memory, conversation compaction, web search, and skill files. All IR components are built from scratch in Python with no external IR libraries. The combination of BM25 and query expansion directly addresses the vocabulary mismatch problem, a foundational challenge in IR, and improves retrieval recall compared to exact keyword matching.

1 Introduction

Large language models have limited context windows and no persistent memory by default. Without IR support, an agent can only reason over information present in the current conversation. This project investigates how classical and modern IR techniques can be integrated into an agent loop to improve context retrieval across sessions and document collections.

IR Agent is built as a minimal but complete agentic system: a hand-written ReAct-style loop (Yao et al., 2023) that dispatches tool calls to a registry of IR components. The system accepts any OpenAI-compatible API, making it compatible with providers such as Berget AI, which hosts open models within the EU.

2 System Architecture

The agent follows a tool-augmented generation loop. On each turn, the agent builds a context from multiple sources, calls the language model with a set of tool schemas, executes any requested tool calls, and repeats until a final text response is produced. The architecture has two layers:

Agent layer. `agent/agent.py` manages the conversation loop and tool dispatch. `agent/context.py` handles conversation compaction when the context window fills up.

Tools layer. A registry (`tools/registry.py`) maps tool names to implementations. New tools can be added by creating one file and registering it, making the system extensible by design.

3 Information Retrieval Methods

3.1 BM25 Document Store

The primary IR contribution is a BM25 document store implemented from scratch in `tools/document_store.py`. BM25 (Best Match 25) is a probabilistic ranking function that extends TF-IDF with term frequency saturation and document length normalization (Robertson and Zaragoza, 2009):

$$score(D,Q) = \sum_{t \in Q} IDF(t) \cdot \frac{f(t,D)(k_1+1)}{f(t,D) + k_1 \cdot norm(D)}$$

$$where norm(D) = 1 - b + b \cdot \frac{|D|}{avgdl}$$

where $k_1 = 1.5$ controls term saturation and $b = 0.75$ controls length normalization. BM25 remains the default ranking function in production search systems such as Elasticsearch and Apache Lucene, and consistently outperforms TF-IDF in standard IR benchmarks (Robertson and Zaragoza, 2009).

Documents are split into overlapping chunks (400 characters, 80-character overlap) before indexing. Chunking enables passage-level retrieval, returning the specific relevant paragraph rather than an entire document. The index persists to disk as a JSON file between sessions.

3.2 Query Expansion

Query expansion addresses the vocabulary mismatch problem: users describe information needs with words that often differ from those used in relevant documents (Furnas et al., 1987). Classical approaches used thesauri or pseudo-relevance feedback; this system uses the language model itself to generate alternative phrasings before searching.

Before querying the BM25 index or memory store, the agent calls the LLM with a structured prompt requesting $n = 3$ alternative phrasings:

> *User query:* "how do cars work?"
> *Expanded:* ["how do cars work?", "automobile engine operation", "vehicle mechanics explained"]

All variants are searched against the BM25 index. Results are merged, deduplicated by chunk ID, and re-ranked by BM25 score. The original query is always preserved. This is applied automatically inside both `document_search` and `memory_search`.

The improvement in recall is measurable: a document containing the word *automobiles* is retrieved when the user queries *cars*, which BM25 alone would miss due to exact term matching.

3.3 Working Memory

The agent maintains a persistent key-value memory store (`tools/memory.py`) serialized as JSON. It supports write, read-all, and keyword search with query expansion. Memory persists across sessions, allowing the agent to accumulate knowledge about user preferences and prior findings over multiple conversations.

3.4 Conversation Compaction

When a conversation exceeds 20 messages, the agent compacts the oldest messages into a plain-text summary saved to disk, retaining only the 8 most recent messages in the active context. The summary is injected as a system message on each subsequent turn. This prevents context window overflow while preserving historical context, inspired by the compaction technique described in the OpenClaw agent framework.

3.5 Web Search and Skill Files

Web search (`tools/web_search.py`) retrieves live information using DuckDuckGo as a no-key fallback, with Brave Search available via API key. Skill files (`skills/`) are markdown documents the agent reads on demand, providing reusable task instructions without modifying the system prompt.

4 Novel Contribution

Compared to OpenClaw, which uses SQLite keyword search for memory retrieval, IR Agent introduces BM25-based document retrieval with chunking and LLM-based query expansion. These features are not present in OpenClaw's built-in tool set. The combination directly targets the vocabulary mismatch problem and demonstrates improved recall on queries where user terminology does not literally appear in indexed documents.

5 Reflection on Working with AI

AI coding tools were used throughout this assignment and proved effective for generating project structure and boilerplate code. However, the experience revealed important limitations requiring active human oversight.

The most significant issue occurred when the AI incorrectly stated that query expansion was already a built-in feature of OpenClaw, which would have undermined the novelty argument of this project. Verifying this claim directly against the OpenClaw documentation showed it was false. This illustrates a key risk of AI-assisted development: the model presents incorrect information confidently, and the developer must verify claims independently.

A second issue arose with documentation: the AI did not automatically update the README when new features were added to the code. After implementing query expansion, the README still described the original system without mentioning it. Keeping documentation in sync required explicitly instructing the AI to update it after each significant change, rather than assuming it would do so proactively.

Overall, AI tools significantly accelerated development, particularly for repetitive tasks. But they function best as a collaborator requiring verification, not as an autonomous authority. Understanding the generated code is essential: without it, errors go undetected and the developer cannot explain or defend their own work.

Links

GitHub: https://github.com/BelenRemedi/ir-agent
Demo Video: https://www.loom.com/share/cfcabd4246dd455b8be805dd956948be

References

George W Furnas, Thomas K Landauer, Louis M Gomez, and Susan T Dumais. 1987. The vocabulary problem in human-system communication. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, pages 221–225.

Stephen Robertson and Hugo Zaragoza. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*.